SMART FACTORY · 회의 자료

Raspberry Pi 5 #1 — 공장 관제(엣지) 서버

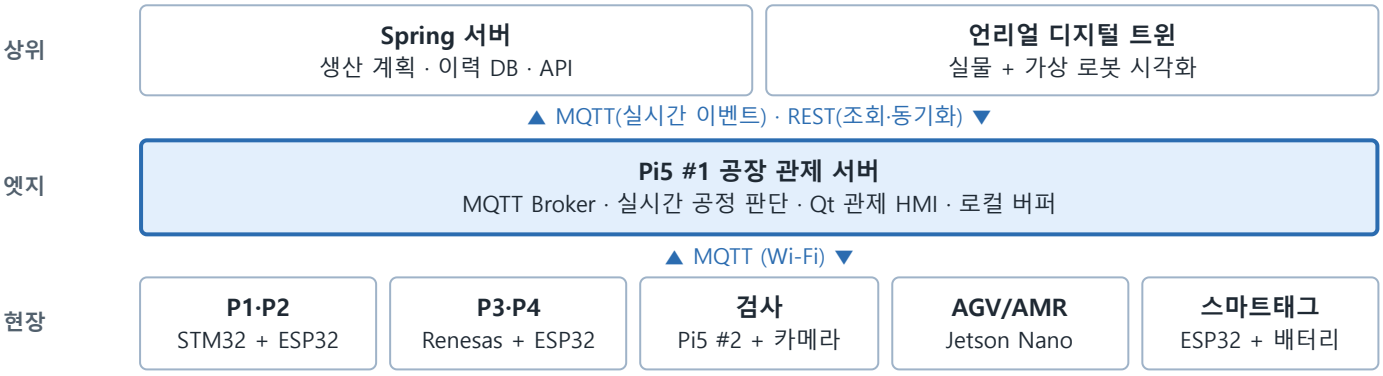
역할 정의 및 Spring 서버 연동 초안 · v0.1 · 2026-11-11 · 확정 전 논의용

> **한 줄 정의** — 공장 현장의 모든 장치(공정 4곳·검사·AGV/AMR·스마트태그)를 MQTT로 묶고, **실시간 판단과 현장 관제**를 맡는 현장 서버. 생산 계획·영구 이력·외부 제공은 상위의 **Spring 서버**가 맡는다.

1. 왜 필요한가

- **명세 필수:** PJT16 — "Raspberry Pi5는 GUI 원격 제어기의 베이스이자 MQTT Broker 역할을 겸한다." (F1601·F1602·F1603 필수, 수정 불가)
- **현장 실시간성:** 공정 허가·이송 명령은 현장 가까이에서 즉시 판단해야 한다. Spring이 잠시 끊겨도 라인은 계속 돈다.
- **역할 분리:** 현장 운영(Pi5) / 업무·데이터(Spring) / 시각화·시뮬레이션(언리얼)을 나눠 담당자별 개발이 가능하다.

2. 시스템 내 위치



3. Pi5 #1이 하는 일 (4가지)

역할	하는 일	구현	명세
① **MQTT Broker**	모든 장치·Spring 사이의 메시지 중계. 토픽 단위 발행/구독, 장치 접속 관리	mosquitto	F1601
② **실시간 공정 판단** (엣지 MES)	RFID 도착 이벤트 수신 → 이전 공정 완료·공정 순서·AGV 정차 확인 → **작업 허가/거부** 응답. 완료 시 다음 목적지를 AGV에, 공정 결과를 스마트태그에 전달. 부품 부족 이벤트 → AMR 출동 명령	Python 서비스 (paho-mqtt)	통합 기능
③ **Qt 관제 HMI**	7인치 LCD 관제 화면. AGV 수동 제어 버튼, AGV 센싱 데이터 실시간 표시, 공정별 상태·차체 위치·AMR 현황·장치 연결 상태	PySide2 (PC는 PySide6로 개발)	F1601 F1602 F1603
④ **로컬 버퍼·장치 감시**	모든 확정 이벤트를 SQLite에 임시 저장 → Spring 장애 시에도 유실 없이 복구 후 재전송. 장치 heartbeat 감시(끊김 감지·표시), 태그 배터리 잔량 수집	SQLite	F1604·F1605 (심화)

4. Spring 서버와의 역할 분담

구분	Pi5 #1 (엣지)	Spring 서버 (상위)
성격	현장 실시간 운영 — "지금 무엇을 해도 되는가"	업무·데이터 관리 — "무엇을 만들고, 무엇이 만들어졌는가"
데이터	진행 중인 작업 상태, 최근 이벤트(임시)	생산 지시, 차체·공정 마스터, **영구 이력 DB**
판단	공정 허가, 다음 목적지, AMR 출동	생산 계획, 통계·리포트, 최종 품질 기록
화면	Qt 관제 HMI (현장 작업자용)	웹 대시보드 / 언리얼에 데이터 제공
장애 시	Spring 없이도 진행 중 작업은 계속 처리	복구 후 엣지 버퍼를 받아 이력 보정

5. Pi5 ↔ Spring 통신

원칙: 실시간으로 흘러야 하는 것은 **MQTT**, 요청-응답으로 주고받는 것은 **REST**. Spring은 Pi5의 Broker에 접속하는 **MQTT 클라이언트**(Spring Integration MQTT / Paho)이자 REST API 서버다.

방향	방식	내용	예시
Spring → Pi5	MQTT	생산 지시 (작업·차체 등록, 투입 공정 순서)	`factory/job/order` {jobId, bodyId, route}
Pi5 → Spring	MQTT	확정 이벤트 (도착·허가·완료·검사 결과·NG 사유)	`factory/edge/event` {eventId, jobId, station, type, ts}
Pi5 → Spring	MQTT	장치 상태 요약 (연결, AGV 모드, 태그 배터리)	`factory/edge/status` (주기 발행)
Pi5 → Spring	REST	**재동기화** — 장애 중 버퍼에 쌓인 이벤트 일괄 업로드	`POST /api/edge/events/batch`
Pi5 → Spring	REST	기동 시 마스터 데이터 동기화 (공정 정의, 태그-차체 매핑)	`GET /api/master/stations`
Qt → Spring	REST	HMI의 과거 이력 조회	`GET /api/bodies/{id}/history`

· 모든 이벤트에 고유 `eventId`를 붙여 Spring에서 중복 저장을 막는다(MQTT 재전송·배치 재업로드 대비). · MQTT는 QoS 1 사용. · 언리얼이 Spring에서 받을지(WebSocket) Broker를 직접 구독할지는 회의에서 결정.

6. 현장 MQTT 토픽 초안

토픽	발행 → 구독	내용
`factory/station/{P1~P4|INSP}/event`	공정 → Pi5	RFID 도착, 작업 완료, 부품 부족, 검사 결과
`factory/station/{id}/cmd`	Pi5 → 공정	작업 허가/거부, LED 상태
`factory/agv/sensing`	AGV → Pi5	PJT16 sensingData (정차 위치·모드 등으로 변경)
`factory/agv/command`	Pi5 → AGV	PJT16 commandData (수동 제어·다음 목적지)
`factory/tag/{tagId}/history`	Pi5 → 태그	공정 결과 기록 지시
`factory/tag/{tagId}/status`	태그 → Pi5	heartbeat, 배터리 잔량
`factory/amr/dispatch`	Pi5 → AMR	부품 공급 출동 (출발지·목적 공정)

7. 흐름 예시 — P2 공정

1. AGV P2 정차 → `agv/sensing` / P2의 STM32가 RFID로 BODY-001 판독 → `station/P2/event`
2. **Pi5:** P1 완료·순서·정차 확인 → `station/P2/cmd` "허가" → P2 LED '작업 중'
3. 작업자 완료 버튼 → `station/P2/event` "완료"
4. **Pi5:** 태그에 "P2 OK" 기록 지시 · AGV에 "다음 P3" · Qt 화면 갱신 · SQLite 저장
5. **Pi5 → Spring:** `edge/event`로 확정 이벤트 전달 → Spring이 영구 이력에 저장 → 언리얼 반영

8. 회의에서 정할 것

- 공정 허가 판단을 Pi5에 둘지, Spring에 둘지 (본 문서는 Pi5 제안)
- AMR 배차 판단 위치 (Pi5 / Spring) 및 부품 부족 감지 방식
- 언리얼 데이터 수신 경로 (Spring WebSocket / MQTT 직접 구독)
- Spring 서버 실행 위치 (PC / EC2 — PJT09 배포 요구와 연결 여부)
- 토픽·JSON 규격 담당자 지정 (모든 파트의 공통 계약)

근거: 16-C-1-16.pdf(PJT16) 3~6쪽 · 본 문서는 설계 제안이며 구현·검증 결과가 아님